

Lakebase Support Ticketing System

A full-stack internal support ticketing application built on **Databricks Lakebase** (managed Postgres OLTP) and deployed as a **Databricks App** — covering schema design, an OAuth-secured data layer, a corporate-grade Gradio UI, and end-to-end smoke testing.

Author	Jayanth Dolai (Senior Software Engineer, UnitedHealth Group / Optum)
Live App URL	https://support-ticketing-7474654640109575.aws.databricksapps.com
GitHub Repo	github.com/demonjd2026-afk/lakebase-support-ticketing
Platform	Databricks Free Edition (AWS) · Lakebase Postgres · Databricks Apps
Frontend	Gradio (Python) — 3-tab corporate ticketing portal
Auth Model	Service-principal OAuth (client_credentials) → Lakebase JWT

Project Highlights

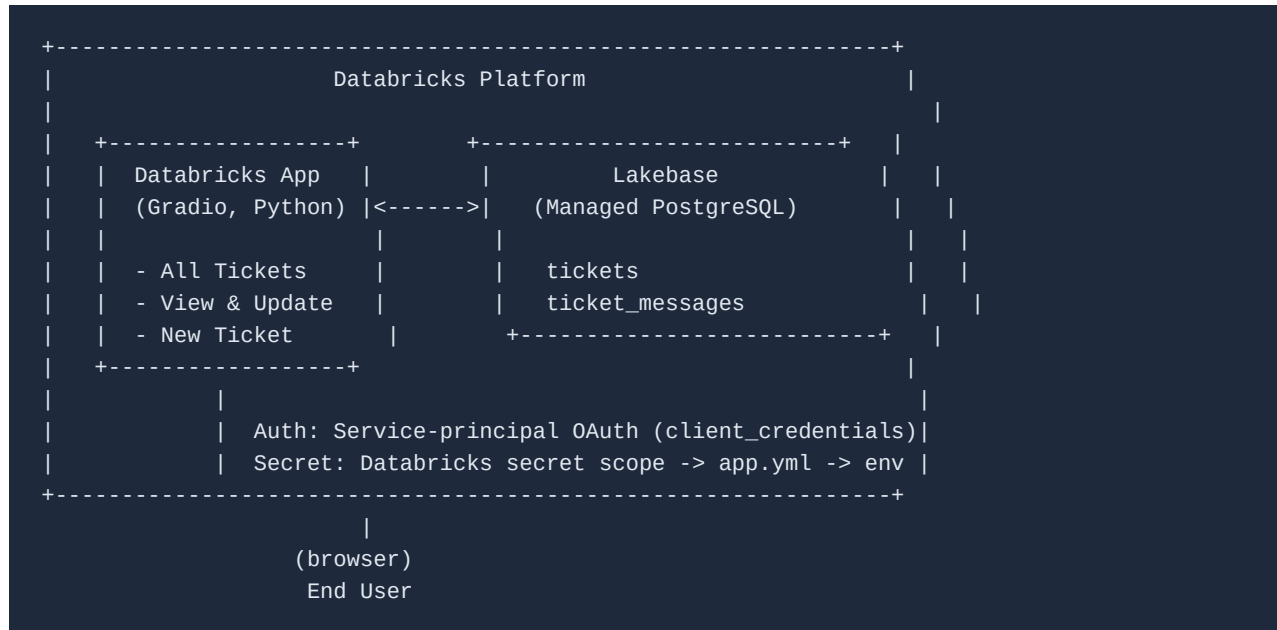
- Two related Postgres tables (**tickets**, **ticket_messages**) with a foreign key and cascading delete
- 4+ seeded tickets across 3 statuses, 11+ threaded messages
- Full CRUD: view, filter, create, update status, add message, delete-with-confirmation
- Bonus features: priority/category fields, live dashboard stats, input validation, styled UI
- Secrets never hit source control — Lakebase credentials come from a Databricks secret scope, injected via **app.yml** and exchanged for a fresh OAuth token on every request

Contents

1. Architecture Overview
2. Phase 1 — Lakebase Schema & Seed Data
3. Phase 2 — Database Access Layer (db.py)
4. Phase 3 — Application UI (app.py, Gradio)
5. Phase 4 — Deployment on Databricks Apps
6. Screenshots — Live Application Walkthrough
7. Bonus Features Implemented
8. Reflection
9. Submission Checklist

1 · Architecture Overview

The application follows a simple three-tier design: a Gradio frontend running as a Databricks App, a thin Python data-access layer, and Lakebase — Databricks' managed PostgreSQL OLTP engine — as the system of record.



Why Lakebase instead of a Delta table?

Dimension	Delta Lake (Analytics)	Lakebase (OLTP)
Designed for	Batch reads, BI, ML	Transactional reads/writes
Latency	Seconds to minutes	Milliseconds
Consistency	Eventually consistent	ACID, row-level
Write pattern	Append / bulk merge	Insert / Update / Delete
Best fit	Dashboards, pipelines	Apps, APIs, operational data

Lakebase Schema & Seed Data

Provisioned a Lakebase project, defined two related Postgres tables, and seeded realistic sample data — all executed directly in the Lakebase SQL Editor.

Lakebase Project Configuration

Project	support-ticketing
Branch	production
Database	databricks_postgres
Auth type	OAuth (Databricks identities & service principals)

Schema DDL

```
CREATE TABLE tickets (  
  ticket_id SERIAL PRIMARY KEY,  
  title TEXT NOT NULL,  
  status TEXT NOT NULL DEFAULT 'open',  
  priority TEXT NOT NULL DEFAULT 'medium',  
  category TEXT,  
  created_by TEXT NOT NULL,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);  
  
CREATE TABLE ticket_messages (  
  message_id SERIAL PRIMARY KEY,  
  ticket_id INT NOT NULL REFERENCES tickets(ticket_id) ON DELETE CASCADE,  
  message_text TEXT NOT NULL,  
  author TEXT NOT NULL,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);  
  
CREATE INDEX idx_messages_ticket_id ON ticket_messages(ticket_id);  
CREATE INDEX idx_tickets_status ON tickets(status);  
CREATE INDEX idx_tickets_priority ON tickets(priority);
```

Seed Data Verification

Sample data was inserted covering all three statuses with multi-message threads, then verified with a join + aggregate query:

```

SELECT t.ticket_id, t.title, t.status, t.priority,
       COUNT(m.message_id) AS message_count
FROM tickets t
LEFT JOIN ticket_messages m ON t.ticket_id = m.ticket_id
GROUP BY t.ticket_id, t.title, t.status, t.priority
ORDER BY t.ticket_id;

```

ID	Title	Status	Priority	Msgs
1	Cluster auto-terminates during pipeline run	open	high	3
2	Unable to access Unity Catalog schema	in_progress	high	3
3	Delta table OPTIMIZE job taking longer	resolved	medium	3
4	Lakebase connection string not recognized	open	medium	2

Result: 4 tickets across 3 distinct statuses, 11 total messages — exceeds the assignment's minimum of 3 tickets / 2 statuses / 2 messages each.

Database Access Layer (db.py)

A single module isolating all SQL. The application code never writes raw queries — every read/write goes through a typed helper function.

Connection Strategy

Inside Databricks Apps, the platform injects `DATABRICKS_CLIENT_ID` and `DATABRICKS_CLIENT_SECRET` for the app's service principal. `db.py` exchanges these for a short-lived OAuth JWT via the `client_credentials` grant, then opens a `psycopg2` connection to Lakebase using that token as the password.

```
def get_oauth_token():
    client_id = os.environ["DATABRICKS_CLIENT_ID"]
    client_secret = os.environ["DATABRICKS_CLIENT_SECRET"]
    host = os.environ.get("DATABRICKS_HOST", ...)

    r = requests.post(f"{host}/oidc/v1/token", data={
        "grant_type": "client_credentials",
        "client_id": client_id,
        "client_secret": client_secret,
        "scope": "all-apis",
    })
    return r.json()["access_token"]

def get_conn():
    token = get_oauth_token()
    client_id = os.environ["DATABRICKS_CLIENT_ID"]
    conn_str = (f"postgresql://{client_id}:{token}"
               f"@{LAKEBASE_HOST}/{LAKEBASE_DB}?sslmode=require")
    return psycopg2.connect(conn_str, cursor_factory=RealDictCursor)
```

Functions Exposed

Function	Purpose
<code>get_all_tickets(status_filter)</code>	List tickets, optional status filter
<code>get_ticket_by_id(id)</code>	Single ticket lookup
<code>get_messages(ticket_id)</code>	Full message thread, chronological
<code>get_stats()</code>	Counts by status for the dashboard
<code>create_ticket(title, created_by, priority, category)</code>	Insert + return new ticket_id
<code>add_message(ticket_id, text, author)</code>	Insert a threaded message
<code>update_status(ticket_id, new_status)</code>	Update ticket status

<code>delete_ticket(ticket_id)</code>	Delete with FK cascade to messages
---------------------------------------	------------------------------------

Application UI (app.py, Gradio)

A three-tab Gradio interface styled to resemble a corporate helpdesk portal — top navigation bar, sticky sidebar dashboard, and content tabs.

UI Structure

- **All Tickets** — filterable table of every ticket, status/priority/category/message-count columns
- **View & Update** — load a ticket by ID; see a formatted detail card, full message thread, status-update control, and a guarded delete action
- **New Ticket** — validated creation form (title, priority, category, requester)
- A sticky left sidebar shows live counts (Total / Open / In Progress / Resolved) that refresh after every write operation

Design System

Light, neutral corporate palette (slate/indigo) modeled on common helpdesk products (Zendesk / Freshdesk / Jira Service Desk): white cards on a soft slate background, colored status chips, Inter typeface, and consistent 42–44px control heights.

Bonus: Delete Confirmation Guard

```
def on_delete_ticket(tid, confirmed):
    if not tid:      return "No ticket loaded", ...
    if not confirmed: return "Tick the confirmation box first", ...
    delete_ticket(int(tid))
    return f"Ticket #{tid} deleted", ...
```


Deployment on Databricks Apps

Connected the GitHub repository to a Databricks App, wired the Lakebase secret as a scoped resource, and verified end-to-end connectivity.

Deployment Configuration

App name	support-ticketing
Source	GitHub — demonjd2026-afk/lakebase-support-ticketing, branch main, path app/
Entrypoint	python app.py (set via app.yml)
Secret resource	Databricks secret scope → LAKEBASE_CONN env var
App URL	https://support-ticketing-7474654640109575.aws.databricksapps.com

app.yml

```
command: ["python", "app.py"]

env:
  - name: LAKEBASE_CONN
    valueFrom: LAKEBASE_CONN
```

Access Control

The App's auto-generated service principal was granted the `databricks_superuser` role on the Lakebase branch via *Roles & Databases*, so the app can read and write `tickets / ticket_messages` using its own OAuth identity — no personal access token stored anywhere.

Smoke Test Results

- ✓ Existing tickets load correctly from Lakebase on page open
- ✓ New ticket created via the form persists and appears in the list
- ✓ Message added to a ticket persists and appears in the thread
- ✓ Ticket status updated and reflected immediately, survives a page refresh
- ✓ Status filter narrows the ticket list correctly
- ✓ Delete requires the confirmation checkbox before removing a ticket

6 · Screenshots — Live Application Walkthrough

Support Ticketing System
Internal IT support portal

Databricks Lakebase
Bootcamp Day 1 - 2026

OVERVIEW

- 5 Total tickets
- 2 Open
- 2 In progress
- 1 Resolved

DATA SOURCE
Databricks Lakebase
Managed PostgreSQL - OLTP

All Tickets | View & Update | New Ticket

Browse and filter every ticket in the support queue.

Filter by status

Refresh

ID	open title	Status	Priority	Category	Created by	Created at	Msgs
5	in_progress Spark job failing with OOM error on large dataset resolved	In Progress	High	infra	jay.dolai	05 Aug 2026, 17:29	1
4	Lakebase connection string not recognized in Databricks App	Open	Medium	infra	jay.dolai	04 Aug 2026, 15:35	2
2	Unable to access Unity Catalog schema after permission update	In Progress	High	access	priya.sharma	03 Aug 2026, 15:35	3
1	Databricks cluster auto-terminates during pipeline run	Open	High	infra	jay.dolai	02 Aug 2026, 15:35	3
3	Delta table OPTIMIZE job taking longer than expected	Resolved	Medium	data	ravi.kumar	31 Jul 2026, 15:35	3

Fig. 1 — All Tickets tab: dashboard sidebar with live counts and the full ticket table.

Support Ticketing System
Internal IT support portal

Databricks Lakebase
Bootcamp Day 1 - 2026

OVERVIEW

- 5 Total tickets
- 2 Open
- 2 In progress
- 1 Resolved

DATA SOURCE
Databricks Lakebase
Managed PostgreSQL - OLTP

All Tickets | **View & Update** | New Ticket

Load a ticket by ID to see its details, history, and take action.

Ticket ID

Load ticket

TICKET DETAILS

Title

Status

Priority

Category

Created by

Created at

MESSAGE THREAD

jay.dolai 02 Aug 2026, 15:35

Fig. 2 — View & Update tab: ticket detail card with status/priority chips and message thread.

The image shows two main panels and a 'DANGER ZONE' section. The 'UPDATE STATUS' panel on the left has a dropdown menu with 'open' selected, and an 'Update status' button. The 'ADD MESSAGE' panel on the right has a 'Your name' field with 'e.g. jay.dolai' and a 'Message' text area with 'Type your reply...'. Below these is a 'Send message' button. The 'DANGER ZONE' section at the bottom left has a checkbox for 'I confirm permanent deletion of this ticket and its messages' and a 'Delete ticket' button.

Fig. 3 — Update Status and Add Message panels, plus the guarded Danger Zone for deletion.

The image shows the 'New Ticket' tab in the 'Support Ticketing System'. The header includes the system name and 'Databricks Lakebase Bootcamp Day 1 - 2026'. The left sidebar shows an 'OVERVIEW' with ticket counts: 5 Total tickets, 2 Open, 2 In progress, and 1 Resolved. The main area has tabs for 'All Tickets', 'View & Update', and 'New Ticket'. Below the tabs is a form titled 'TICKET INFORMATION' with fields for 'Title *', 'Priority *', 'Category', and 'Your name / email *'. The 'Title' field has a placeholder 'Briefly describe the issue...' and a dropdown for 'low', 'medium', and 'high'. The 'Priority' field has a dropdown with 'medium' selected. The 'Category' field has a placeholder 'e.g. infra, access, data, billing'. The 'Your name / email' field has a placeholder 'e.g. jay.dolai'.

Fig. 4 — New Ticket tab: validated creation form (title, priority, category, requester).

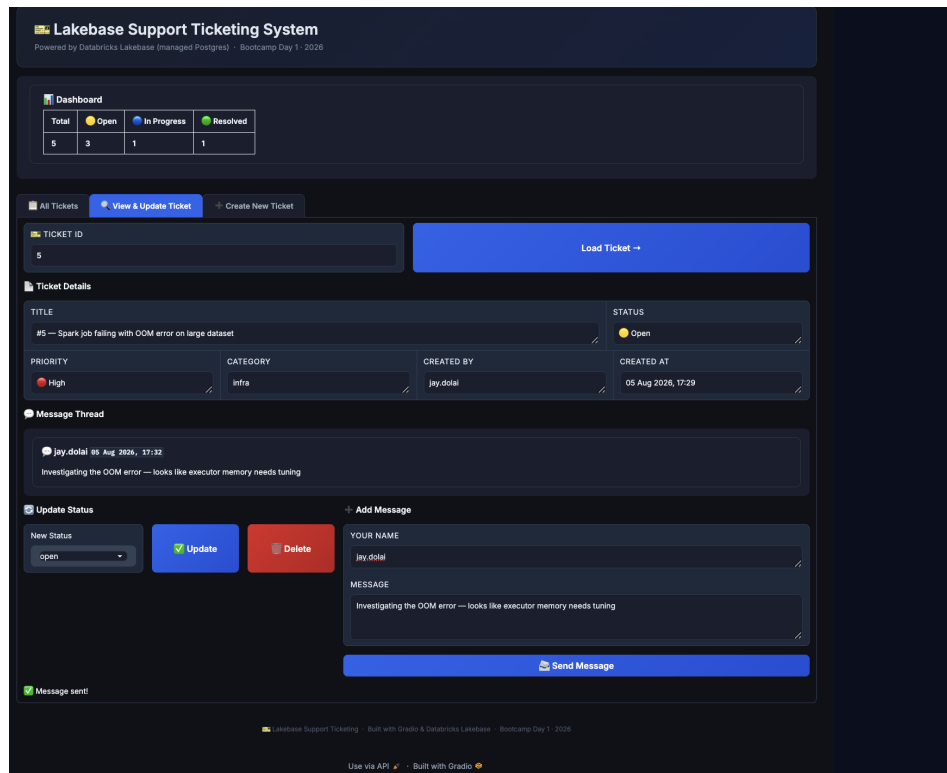


Fig. 5 — Message successfully sent and appended to the ticket's thread.

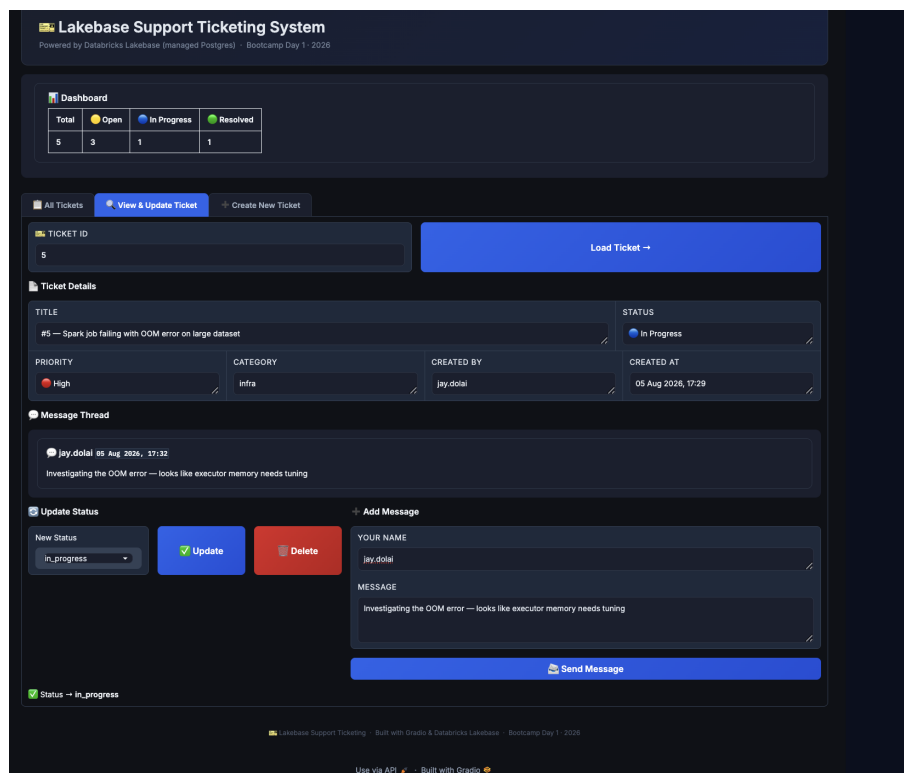


Fig. 6 — Ticket status updated to in_progress, confirmed with inline feedback.

7 · Bonus Features Implemented

- ✓ **Priority & category** — both fields present in schema, forms, and table columns
- ✓ **Filter by status** — dropdown on the All Tickets tab (All / open / in_progress / resolved)
- ✓ **Input validation** — required-field checks with inline error messages on create/update/message actions
- ✓ **Ticket statistics** — live sidebar dashboard (Total / Open / In Progress / Resolved), refreshed after every mutation
- ✓ **Delete with confirmation** — explicit checkbox required before the delete action is accepted
- ✓ **Improved visual design** — corporate helpdesk-style UI with custom CSS, status/priority chips, and a sticky dashboard sidebar

8 · Reflection

What was the most difficult part?

Getting the Databricks App to authenticate against Lakebase. Lakebase's OAuth connections require a JWT access token — not a personal access token — so the working path was to have the app's own service principal exchange its injected `DATABRICKS_CLIENT_ID/DATABRICKS_CLIENT_SECRET` for a JWT via the OIDC `client_credentials` grant, and to grant that principal a Lakebase role on the branch. Several intermediate approaches (raw PAT, hardcoded connection strings, mismatched hostnames) failed with distinct errors before the correct service-principal-OAuth flow was identified.

How is Lakebase different from a traditional analytics table?

Lakebase is managed PostgreSQL exposed over the standard wire protocol, giving row-level ACID transactions and millisecond read/write latency — exactly what an interactive app doing single-row inserts and updates needs. A Delta table is optimized for large, append-oriented batch writes and file-level commits; using it for a live app's create/update/delete operations would be both slow and awkward compared to Lakebase's transactional model.

What feature would you add next?

An AI agent layer on top of the same Lakebase tables — using Databricks Model Serving (e.g. Llama or Claude) to auto-triage new tickets (suggest priority/category), draft reply messages, and summarize a long thread on load, mirroring the agentic pattern already used in the AI Stock Market Research Assistant capstone.

9 · Submission Checklist

- ✓ Databricks App URL — <https://support-ticketing-7474654640109575.aws.databricksapps.com>
- ✓ Source code — github.com/demonjd2026-afk/lakebase-support-ticketing
- ✓ Screenshot of the deployed application — included above
- ✓ Screenshot of the Lakebase tables and sample records — included above
- ✓ Reflection (3–5 sentences) — included above